National Central University • Department of Computer Science & Information Engineering

CO3020 Computer Organization • Midterm Mock Examination [Official Solution & Detailed Grading Rubric]

Student ID	OFFICIAL ANSWER KEY	**Candidate Name**	CSIE Instructional Staff
Exam Duration	100 Minutes (Closed-book, electronic devices strictly prohibited)	**Points / Problems**	100 Points / 8 Multi-part Problems (22 Sub-questions)

Exam Instructions:
1. This comprehensive examination synthesizes **Assignment 1 numerical calculations** and the **instructor-designated Chapter 3 core textbook problems** (Exercises 3.12, 3.13, 3.18, 3.27, 3.29, 3.35–3.37).
2. All calculation and design problems must include clear, step-by-step mathematical derivations and parameter substitutions. Numerical answers provided without intermediate derivation steps will receive zero credit.
3. Unless explicitly instructed otherwise, round all fractional results to two decimal places. Express machine instructions in standard 32-bit binary or hexadecimal notation (prefixed with `0x`).
4. Floating-point computations must strictly adhere to the IEEE 754 standard and the Round to Nearest Even mode.

Part I: Computer Abstractions, Performance & The Power Wall (Chapter 1) [Weight: 25 Points]

[10 Points] Question 1: Semiconductor Scaling, Moore's Law Doubling Time, and CMOS Dynamic Power Dissipation [Aligned with HW1 Q1]

Consider the desktop microprocessor process technology metrics from Intel between 2012 and 2020:

- **2012 (Ivy Bridge Architecture)**: Process feature size $t_1 = 22\ \text{nm}$, base clock frequency $f_1 = 3.4\ \text{GHz}$, supply voltage $V_1 = 1.20\ \text{V}$.
- **2020 (Tiger Lake Architecture)**: Process feature size $t_2 = 10\ \text{nm}$, base clock frequency $f_2 = 4.8\ \text{GHz}$, supply voltage $V_2 = 0.95\ \text{V}$.

Assume that the silicon die size remains constant across both microarchitectures, and that the total transistor density on the chip scales inversely with the square of the process feature size t: $\text{Density} \propto (1/t)^2$.

(a) (4 pts) Compute the transistor density scaling ratio from 2012 to 2020 (spanning 8 years), and determine the Compound Annual Growth Rate (CAGR) as a percentage (%).

[Official Solution & Rubric] Rubric: Scaling ratio 2 pts, CAGR derivation & value 2 pts

1. **Transistor Density Scaling Ratio**:

$$\text{Density Ratio} = \left(\frac{t_1}{t_2}\right)^2 = \left(\frac{22\ \text{nm}}{10\ \text{nm}}\right)^2 = 2.2^2 = \mathbf{4.84}\times$$

2. **Compound Annual Growth Rate (CAGR over 8 years)**:

$$\text{CAGR} = (\text{Density Ratio})^{1/8} - 1 = (4.84)^{0.125} - 1 = 1.2178 - 1 = 0.2178 = \mathbf{21.78\%}$$

(b) (3 pts) Based on the calculated CAGR, determine the doubling time (T_{double}, in years) required for the microprocessor's transistor count to double.

[Official Solution & Rubric] Rubric: Formula 1 pt, Numerical calculation 2 pts

Method 1 (via standard CAGR definition):

$$T_{\text{double}} = \frac{\ln(2)}{\ln(1+\text{CAGR})} = \frac{\ln(2)}{\ln(1.2178)} = \frac{0.69315}{0.19705} = \mathbf{3.52}\ \text{years}$$

Method 2 (Instructor's direct logarithmic formula):
Since transistor count scales with $(1/t)^2$, the 8-year density ratio is $(t_1/t_2)^2$:

$$T_{\text{double}} = \frac{n \ln 2}{2\ln(t_1/t_2)} = \frac{8\ln 2}{2\ln(22/10)} = \frac{4\ln 2}{\ln(2.2)} = \frac{4 \times 0.69315}{0.78846} = \mathbf{3.52}\ \text{years}$$

Conclusion: Microprocessor transistor density doubles approximately every **3.52 years** (a noticeable deceleration compared to early Moore's Law pace of 18–24 months).

(c) (3 pts) The CMOS dynamic switching power formula is defined as $P = C \times V^2 \times f$. Assume that due to gate insulator improvements, the effective capacitive load C in 2020 is reduced by 20% compared to 2012 ($C_{2020} = 0.80 \times C_{2012}$). Compute the dynamic power ratio P_{2020}/P_{2012}, and state explicitly whether the dynamic power increased or decreased.

[Official Solution & Rubric] Rubric: Formula substitution 1 pt, Ratio value 1 pt, Increase/Decrease verdict 1 pt

$$\frac{P_{2020}}{P_{2012}} = \frac{C_{2020} \times V_{2020}^2 \times f_{2020}}{C_{2012} \times V_{2012}^2 \times f_{2012}} = 0.80 \times \left(\frac{0.95}{1.20}\right)^2 \times \left(\frac{4.8}{3.4}\right)$$

$$\frac{P_{2020}}{P_{2012}} = 0.80 \times 0.62674 \times 1.41176 = \mathbf{0.7078} \approx \mathbf{70.78\%}$$

Conclusion: The dynamic power in 2020 is approximately **70.78%** of that in 2012 (dynamic power **decreased by 29.22%**). This highlights the quadratic benefit of supply voltage reduction in curtailing dynamic power dissipation.

[15 Points] Question 2: CPU Performance Equation, Multi-Class Weighted Average CPI, Compiler Optimization, and Amdahl's Law Physical Limit [Aligned with HW1 Q2, Q3, Q4]

A benchmark application has a dynamic instruction count of $IC = 2.0 \times 10^9$ instructions. The instruction mix breakdown and per-class CPI for two candidate processor implementations are given in the table below:

Instruction Class	Proportion of Total IC (%)	Processor P1 CPI (Clock: 2.5 GHz)	Processor P2 CPI (Clock: 3.2 GHz)
Class A (ALU Operations)	40%	1	2
Class B (Conditional Branches)	20%	2	2
Class C (Load / Store)	30%	4	3
Class D (Floating-Point)	10%	5	4

(a) (4 pts) Calculate the global weighted average CPI for both Processor P1 and Processor P2. Based on total CPU execution time, determine which processor executes the benchmark faster and compute the performance speedup factor.

[Official Solution & Rubric] Rubric: CPI for P1/P2 1 pt each, Execution times 1 pt, Speedup & faster CPU 1 pt

1. **Weighted Average CPI Calculation**:

$$\text{CPI}_{P1} = (0.40 \times 1) + (0.20 \times 2) + (0.30 \times 4) + (0.10 \times 5) = 0.40 + 0.40 + 1.20 + 0.50 = \mathbf{2.50}$$

$$\text{CPI}_{P2} = (0.40 \times 2) + (0.20 \times 2) + (0.30 \times 3) + (0.10 \times 4) = 0.80 + 0.40 + 0.90 + 0.40 = \mathbf{2.50}$$

2. **CPU Execution Time and Comparison**:

$$T_{P1} = \frac{IC \times \text{CPI}_{P1}}{f_1} = \frac{2.0 \times 10^9 \times 2.50}{2.5 \times 10^9 \text{ Hz}} = \mathbf{2.000} \text{ s}$$

$$T_{P2} = \frac{IC \times \text{CPI}_{P2}}{f_2} = \frac{2.0 \times 10^9 \times 2.50}{3.2 \times 10^9 \text{ Hz}} = \mathbf{1.5625} \text{ s}$$

Conclusion: **Processor P2 is faster**. Speedup $\text{Speedup} = \frac{T_{P1}}{T_{P2}} = \frac{2.000}{1.5625} = \mathbf{1.28}\times$.

(b) (5 pts) Compiler Optimization Impact: A newly engineered optimizing compiler, Compiler Y, restructures loop memory scheduling and successfully **reduces the number of Class C (Load/Store) instructions by 50%**, while leaving the dynamic counts of Classes A, B, and D unaffected. When executing on Processor P1, calculate: (i) the new total dynamic instruction count, (ii) the new weighted average CPI, (iii) the new execution time, and (iv) the speedup relative to the baseline compiler.

[Official Solution & Rubric] Rubric: New IC & cycles 2 pts, New CPI & execution time 2 pts, Speedup 1 pt

1. **Baseline Instruction Counts**: $IC_A = 0.8 \times 10^9, IC_B = 0.4 \times 10^9, IC_C = 0.6 \times 10^9, IC_D = 0.2 \times 10^9$.
2. **After Reducing Class C by 50%**: $IC'_C = 0.50 \times 0.6 \times 10^9 = 0.3 \times 10^9$.

New total dynamic instruction count: $IC_{\text{new}} = (0.8 + 0.4 + 0.3 + 0.2) \times 10^9 = \mathbf{1.70 \times 10^9}$ instructions.

3. **Total CPU Clock Cycles on Processor P1**:

$$\text{Cycles}_{\text{new}} = (0.8 \times 10^9 \times 1) + (0.4 \times 10^9 \times 2) + (0.3 \times 10^9 \times 4) + (0.2 \times 10^9 \times 5) = 3.80 \times 10^9 \text{ cycles}$$

4. **New Weighted CPI and Execution Time**:

$$\text{CPI}_{\text{new}} = \frac{3.80 \times 10^9}{1.70 \times 10^9} \approx \mathbf{2.235}$$

$$T_{\text{new}} = \frac{\text{Cycles}_{\text{new}}}{f_1} = \frac{3.80 \times 10^9}{2.5 \times 10^9} = \mathbf{1.520} \text{ s}$$

5. **Speedup Relative to Baseline Compiler**:

$$\text{Speedup} = \frac{T_{\text{old}}}{T_{\text{new}}} = \frac{2.000 \text{ s}}{1.520 \text{ s}} \approx \mathbf{1.316}\times$$

(c) (6 pts) Amdahl's Law and Single-Component Physical Limit: Consider the baseline program running on Processor P1 (baseline execution time $T_0 = 2.0$ seconds). Engineering management proposes: **"Focus microarchitectural redesign exclusively on optimizing the branch execution unit (Class B) to reduce total program execution time by 25%."**

(i) Calculate the total execution time spent exclusively on branch instructions in the baseline program.

(ii) If branch execution time is reduced to its theoretical physical limit (0 seconds), what is the maximum possible execution time reduction achievable for the overall application?

(iii) Using rigorous mathematical derivation based on Amdahl's Law, determine whether this 25% execution time reduction target is physically attainable. Clearly state your verdict (Attainable or Physically Unattainable) and justify.

[Official Solution & Rubric] Rubric: Branch time (i) 2 pts, Limit reduction (ii) 2 pts, Amdahl proof & verdict (iii) 2 pts

1. **Baseline Branch Execution Time**:

$$T_{\text{Branch}} = \frac{IC_B \times \text{CPI}_B}{f_1} = \frac{0.40 \times 10^9 \times 2}{2.5 \times 10^9 \text{ Hz}} = \mathbf{0.320} \text{ s}$$

(Branch instructions represent $0.320/2.000 = 16.0\%$ of total execution time).

2. **Theoretical Limit of Execution Time Reduction**:

If branch execution latency is reduced to zero ($T'_{\text{Branch}} = 0$), the maximum possible time saved is exactly the entire branch portion:

$$\Delta T_{\max} = T_{\text{Branch}} - 0 = \mathbf{0.320} \text{ s}$$

3. **Target Feasibility Proof via Amdahl's Law**:

• The management target requires saving 25% of total execution time:

$$\Delta T_{\text{target}} = 2.000 \text{ s} \times 25\% = \mathbf{0.500} \text{ s}$$

• By Amdahl's Law, the performance enhancement is strictly bounded by the fraction of time affected:

$$\text{Execution Time}_{\text{enhanced}} = T_{\text{unaffected}} + \frac{T_{\text{affected}}}{S} \geq T_{\text{unaffected}} = 2.000 - 0.320 = 1.680 \text{ s}$$

• Because $\Delta T_{\max} = 0.320 \text{ s} < \Delta T_{\text{target}} = 0.500 \text{ s}$, the targeted time reduction exceeds the entire physical runtime of the branch subsystem.

Verdict: **Physically Unattainable** (No degree of branch optimization, even infinite speedup $S \to \infty$, can achieve a 25% overall execution time reduction).

Part II: Instructions: Language of the Computer (Chapter 2) [Weight: 25 Points]

[10 Points] Question 3: MIPS Instruction Format Deconstruction, Field Binary Expansion, and 32-Bit Hexadecimal Machine Code Assembly [Aligned with HW1 Q6]

Examine the hardware mapping between MIPS assembly instructions and 32-bit hardware register fields.

(a) (5 pts) R-type Instruction Encoding: Assemble the MIPS assembly instruction **`add $s2, $t0, $t1`** into 32-bit machine code.
Given register numbers: **`$t0 = $8, $t1 = $9, $s2 = $18`**, and arithmetic function code **`funct = 32`**.
(i) Identify the 6 instruction fields and their decimal values in hardware encoding order (note the register operand layout!): **`op, rs, rt, rd, shamt, funct`**.
(ii) Expand each field into its binary bit sequence.
(iii) Output the final 32-bit machine instruction as an 8-digit hexadecimal value (e.g., **`0x01099020`**).

[Official Solution & Rubric] Rubric: Field layout 2 pts, Binary bitstring 2 pts, Hexadecimal code 1 pt

1. **Field Breakdown (Decimal)**:
• op (6 bits) = 0 (R-type arithmetic)
• `rs` (5 bits) = 8 (First source operand `$t0`)
• `rt` (5 bits) = 9 (Second source operand `$t1`)
• `rd` (5 bits) = 18 (Destination register `$s2`)
• `shamt` (5 bits) = 0 (Not a shift instruction)
• `funct` (6 bits) = 32 (Function code for add)
2. **Binary Expansion**:
`000000 01000 01001 10010 00000 100000`
3. **Hexadecimal Machine Code**:
Group into 4-bit nibbles: `0000 0001 0000 1001 1001 0000 0010 0000` ⇒ **0x01099020**.

(b) (5 pts) I-type Instruction Disassembly and Branch Target Addressing: Given the hardware instruction fields below:
• **`op = 0x23`** (hexadecimal)
• **`rs = 29`** (stack pointer **`$sp`**)
• **`rt = 31`** (return address **`$ra`**)
• **`immediate = 8`** (decimal)
(i) Write the corresponding standard MIPS assembly instruction syntax.
(ii) Write its 32-bit hexadecimal machine code.
(iii) Extension: Suppose a conditional branch instruction **`beq $s0, $s1, 16`** (where **`immediate = 16`**) is located at byte memory address **`0x00400030`**. Calculate the exact hexadecimal target memory address (Target PC) taken when the branch evaluates to true.

[Official Solution & Rubric] Rubric: Assembly syntax 1 pt, Hex machine code 2 pts, Target PC calculation 2 pts

1. **MIPS Assembly Instruction Syntax**:
Opcode `0x23` (35 decimal) corresponds to load word `lw`, with base register `rs = $sp`, target register `rt = $ra`, and offset 8: **`lw $ra, 8($sp)`**.
2. **Hexadecimal Machine Code Assembly**:
• op (6 bits) = `100011` (0x23)
• `rs` (5 bits) = `11101` (29)
• `rt` (5 bits) = `11111` (31)
• `immediate` (16 bits) = `0000 0000 0000 1000` (8)
Full binary: `100011 11101 11111 0000000000001000` ⇒ **0x8FBF0008**.
3. **PC-Relative Branch Target Address Calculation**:

$$\text{Target PC} = (\text{PC} + 4) + (\text{immediate} \times 4)$$

Base address: $\text{PC} + 4 = 0\text{x}00400030 + 4 = 0\text{x}00400034$.
Byte offset: $\text{Offset} = 16 \times 4 = 64 \text{ bytes} = 0\text{x}00000040$.

$$\text{Target PC} = 0\text{x}00400034 + 0\text{x}0040 = \mathbf{0x00400074}$$

[15 Points] Question 4: Loop Reverse Engineering, Closed-Form Dynamic Instruction Count, and Strided 2D Array Addressing [Aligned with HW1 Q7, Q9]

Analyze the following MIPS assembly loop segment (assume delayed branching is disabled and integer arithmetic does not overflow):

```
# Initial state: t1 contains non-negative integer N (N >= 0), s2 is initialized to 0
LOOP:
    slt  t2, 0, $t1       # Step 1: Evaluate 0 < $t1
    beq  t2, 0, DONE      # Step 2: Exit loop if condition is false
    addi t1, t1, -1       # Step 3: Decrement loop counter by 1
    addi s2, s2, 3        # Step 4: Accumulate sum by adding 3
    j    LOOP             # Step 5: Unconditional jump back to LOOP
DONE:
```

(a) (7 pts) Loop Reverse Engineering and Dynamic Instruction Counting:

(i) (2 pts) Reverse engineer the assembly loop into equivalent high-level C code (use variable `i` for `$t1` and `B` for `$s2`).

(ii) (2 pts) If register `$t1` is initialized to $N = 12$, what is the final value stored in `$s2` upon reaching `DONE`?

(iii) (3 pts) Derive a closed-form mathematical formula for the total dynamic basic MIPS instructions executed from entering `LOOP` until final exit to `DONE` for any initial integer $N \geq 0$ (including the final termination evaluation). Calculate the exact dynamic instruction count for $N = 12$.

[Official Solution & Rubric] Rubric: C code 2 pts, Final value 2 pts, Formula & value for N=12 3 pts

1. **Equivalent C Code**:

• Standard concise idiom: `while (i > 0) { i--; B += 3; }` (or `for (; i > 0; i--) { B += 3; }`).

• Instructor's canonical key idiom (explicitly capturing boolean temporary variable `temp` in `$t2`):

```
while ((temp = (i > 0))) { i--; B += 3; }
```

(Both implementations are logically and semantically identical; both receive full credit).

2. **Final Accumulator Value for $N = 12$**:

The loop iterates exactly 12 times, incrementing $s2 by 3 each time$:s2 = 12 \times 3 = \mathbf{36}$.

3. **Closed-Form Dynamic Instruction Count Derivation**:

• Each completed iteration executes 5 instructions: `slt`, `beq` (not taken), `addi`, `addi`, `j`.

• When $t1 reaches 0, the final termination check executes 2 instructions: `slt` ($0 < 0$ evaluates to false, setting $t2 = 0$) and `beq` (taken, branching to `DONE`).

• Closed-form formula:

$$\text{Total Instructions} = \mathbf{5N + 2}$$

• For $N = 12$: $\text{Total Instructions} = 5(12) + 2 = \mathbf{62}$ instructions.

(b) (8 pts) Strided Array Access `D[4*j]` Byte Offset and Assembly Implementation:
Consider the following nested C loop:

```
for (i = 0; i < a; i++) {
    for (j = 0; j < b; j++) {
        D[4 * j] = i + j;
    }
}
```

Assume integer variables `a, b, i, j` reside in registers `$s0, $s1, $t0, $t1`. The base memory address of array `D` resides in `$s2`. Each array element is a 32-bit signed integer (4 bytes), word-aligned, and does not cause overflow.
(i) (3 pts) Rigorously derive the formula for the **memory byte offset** of element `D[4*j]` relative to base address `$s2`. If logical shift left (`sll`) is utilized to compute this multiplication, what shift amount (`shamt`) must be configured?
(ii) (5 pts) Write complete, highly optimized MIPS assembly code implementing this nested loop using the minimum number of instructions. Maintain the initial values of `$s0, $s1, $s2` intact.

[Official Solution & Rubric] Rubric: Byte offset derivation & shamt 3 pts, Optimized MIPS code 5 pts

1. **Byte Offset Derivation**:
• The logical element index is $4 \times j$.
• Since each integer element occupies 4 bytes in word-aligned memory:

$$\text{Byte Offset} = (4 \times j) \times 4 \text{ bytes} = 16 \times j = 2^4 \times j$$

• Multiplying by 16 is equivalent to shifting left by 4 bits. Thus, the shift amount must be configured to **shamt = 4** (instruction: `sll $t2, $t1, 4`).
2. **Instructor's Canonical 12-Instruction Optimal Implementation**:
Leveraging the non-negative properties of loop bounds ($a, b \geq 0$), this canonical architecture checks for zero upfront and utilizes bottom conditional branches (bne) to eliminate internal `slt` tests and unconditional jumps (`j`). This achieves the minimal possible static instruction count of only 12 instructions, with just 6 instructions per inner loop iteration:

```
        addi t0,zero, 0      # i = 0
        beq  s0,zero, DONE   # If a == 0, skip outer loop entirely
OUTER:  addi t1,zero, 0      # j = 0
        beq  s1,zero, NEXT   # If b == 0, skip inner loop
INNER:  sll  t2,t1, 4        # t2 = 16 * j (byte offset) add t2, s2,t2      # $t2 = &D[4*j] (reuses $t2 to
conserve registers)
        add  t3,t0, $t1      # $t3 = i + j
        sw   t3, 0(t2)       # Memory[D + 16*j] = i + j
        addi t1,t1, 1        # j++
        bne  t1,s1, INNER    # If j != b, repeat inner loop (single branch decision)
NEXT:   addi t0,t0, 1        # i++
        bne  t0,s0, OUTER    # If i != a, repeat outer loop
DONE:
```

Note (Alternative Acceptable Solution): An implementation using standard top-testing `slt + beq` and bottom `j` (14 total instructions) is semantically valid and achieves full functional credit, but the 12-instruction bottom-testing loop is the strict minimal-instruction baseline from the instructor's official key.

Part III: Computer Arithmetic & Floating-Point Systems (Chapter 3) [Weight: 50 Points]

[15 Points] Question 5: Baseline Multiplier vs. Optimized Multiplier 6-Cycle Hardware Trace & Verification [Aligned with Exercises 3.12, 3.13]

Compute the multiplication of two 6-bit unsigned integers:

- **Multiplicand (M)**: Octal $45_8 = 37_{10} = 100101_2$.
- **Multiplier (Q)**: Octal $13_8 = 11_{10} = 001011_2$.

Expected mathematical product: $37 \times 11 = 407_{10} = 0001\ 1001\ 0111_2 = 627_8$.

(a) (8 pts) Baseline Multiplier Hardware Trace (Textbook Figure 3.3 Architecture):
Hardware register setup: Multiplicand is placed in a **12-bit register** (initial value in the lower 6 bits: **`0000 0010 0101`**, shifted left by 1 bit each cycle); Multiplier is placed in a **6-bit register** (initial value: **`001011`**, shifted right by 1 bit each cycle); Product is placed in a **12-bit register** (initial value: **`0000 0000 0000`**).
Provide the complete 6-cycle execution trace table from Iteration 0 through Iteration 6.

[Official Solution & Rubric] Rubric: Initial state 1 pt, Cycles 1–6 traces 6 pts, Final product verification 1 pt

Iteration	Step (Hardware Action)	Multiplier (6-bit)	Multiplicand (12-bit)	Product (12-bit)
0	Initial values	001011	0000 0010 0101	0000 0000 0000
1	1: LSB=1 ⇒ Prod = Prod + M	001011	0000 0010 0101	0000 0010 0101
	2: Shift Left Multiplicand	001011	0000 0100 1010	0000 0010 0101
	3: Shift Right Multiplier	000101	0000 0100 1010	0000 0010 0101
2	1: LSB=1 ⇒ Prod = Prod + M	000101	0000 0100 1010	0000 0110 1111
	2: Shift Left Multiplicand	000101	0000 1001 0100	0000 0110 1111
	3: Shift Right Multiplier	000010	0000 1001 0100	0000 0110 1111
3	1: LSB=0 ⇒ No addition	000010	0000 1001 0100	0000 0110 1111
	2: Shift Left Multiplicand	000010	0001 0010 1000	0000 0110 1111
	3: Shift Right Multiplier	000001	0001 0010 1000	0000 0110 1111
4	1: LSB=1 ⇒ Prod = Prod + M	000001	0001 0010 1000	0001 1001 0111
	2: Shift Left Multiplicand	000001	0010 0101 0000	0001 1001 0111
	3: Shift Right Multiplier	000000	0010 0101 0000	0001 1001 0111
5	1: LSB=0 ⇒ No addition	000000	0010 0101 0000	0001 1001 0111
	2: Shift Left Multiplicand	000000	0100 1010 0000	0001 1001 0111
	3: Shift Right Multiplier	000000	0100 1010 0000	0001 1001 0111
6	1: LSB=0 ⇒ No addition	000000	0100 1010 0000	0001 1001 0111
	2: Shift Left Multiplicand	000000	1001 0100 0000	0001 1001 0111
	3: Shift Right Multiplier	000000	1001 0100 0000	0001 1001 0111

Final 12-bit product: **$0001\ 1001\ 0111_2$** = 407_{10} = **627_8**.

(b) (7 pts) Optimized Multiplier Hardware Trace (Textbook Figure 3.5 Architecture):
The multiplicand is held stationary in a **6-bit register** ($M = 100101_2$); Product and Multiplier share a single **12-bit register [A | Q]**, where the upper 6 bits A are initialized to **000000**, and the lower 6 bits Q hold the multiplier **001011**. In each iteration: if $Q_0 = 1$, compute $A = A + M$, then shift the **entire 12-bit register [A | Q] right by 1 bit**. Record the value of [A | Q] after addition and after shifting for each of the 6 cycles, and verify the final 12-bit output.

[Official Solution & Rubric] Rubric: Initial register layout 1 pt, 6-cycle trace 5 pts, Final verification 1 pt

Iteration	Step (Hardware Action)	Multiplicand (M)	Combined Register [A | Q] (12-bit)
0	Initial values (A=0, Q=Multiplier)	100101	000000 001011
1	Q_0=1 ⇒ A = A + M	100101	100101 001011
	Shift Right [A | Q] 1 bit	100101	**010010 100101**
2	Q_0=1 ⇒ A = A + M (010010 + 100101 = 110111)	100101	110111 100101
	Shift Right [A | Q] 1 bit	100101	**011011 110010**
3	Q_0=0 ⇒ No addition	100101	011011 110010
	Shift Right [A | Q] 1 bit	100101	**001101 111001**
4	Q_0=1 ⇒ A = A + M (001101 + 100101 = 110010)	100101	110010 111001
	Shift Right [A | Q] 1 bit	100101	**011001 011100**
5	Q_0=0 ⇒ No addition	100101	011001 011100
	Shift Right [A | Q] 1 bit	100101	**001100 101110**
6	Q_0=0 ⇒ No addition	100101	001100 101110
	Shift Right [A | Q] 1 bit	100101	**000110 010111**

Final 12-bit register output: A = 000110, Q = 010111 ⇒ **$0001\ 1001\ 0111_2$** = 407_{10} = **627_8**.

[10 Points] **Question 6: Restoring Division 6-Cycle Hardware Trace & Signed Division Sign Rules [Aligned with Exercise 3.18 & Slides]**

Compute the division of two 6-bit unsigned integers:

- **Dividend (D)**: Octal $62_8 = 50_{10} = 110010_2$.
- **Divisor (d)**: Octal $12_8 = 10_{10} = 001010_2$.

(a) (7 pts) Restoring Division Step-by-Step Hardware Trace (Textbook Figure 3.8 / 3.10 Architecture):
Divisor is fixed in a 6-bit register (**`001010`**); Remainder and Quotient share a **12-bit register [Remainder | Quotient]**, initialized with the upper 6 bits set to **`000000`** and lower 6 bits holding the dividend **`110010`**.
Per-cycle sequence: (1) Shift [Remainder | Quotient] left by 1 bit; (2) Remainder = Remainder - Divisor; (3) Evaluation: If Remainder < 0, restore remainder (add Divisor back) and set quotient LSB $q_0 = 0$; if Remainder ≥ 0, keep remainder and set quotient LSB $q_0 = 1$.
Complete the full 6-cycle trace table, and state the final Quotient and Remainder.

[Official Solution & Rubric] Rubric: Cycles 1–6 trace 5 pts, Final Quotient and Remainder 2 pts

Iteration	Step (Hardware Action)	Divisor	Remainder (6-bit)	Quotient (6-bit)
0	Initial values	001010	000000	110010
1	1: Shift Left [Rem | Quot]	001010	000001	10010_
	2&3: Rem - Div = 1 - 10 = -9 < 0 ⇒ Restore, q_0=0	001010	**000001**	**100100**
2	1: Shift Left [Rem | Quot]	001010	000011	00100_
	2&3: Rem - Div = 3 - 10 = -7 < 0 ⇒ Restore, q_0=0	001010	**000011**	**001000**
3	1: Shift Left [Rem | Quot]	001010	000110	01000_
	2&3: Rem - Div = 6 - 10 = -4 < 0 ⇒ Restore, q_0=0	001010	**000110**	**010000**
4	1: Shift Left [Rem | Quot]	001010	001100 (12)	10000_
	2&3: Rem - Div = 12 - 10 = 2 ≥ 0 ⇒ No Restore, q_0=1	001010	**000010**	**100001**
5	1: Shift Left [Rem | Quot]	001010	000101 (5)	00001_
	2&3: Rem - Div = 5 - 10 = -5 < 0 ⇒ Restore, q_0=0	001010	**000101**	**000010**
6	1: Shift Left [Rem | Quot]	001010	001010 (10)	00010_
	2&3: Rem - Div = 10 - 10 = 0 ≥ 0 ⇒ No Restore, q_0=1	001010	**000000**	**000101**

Final Division Results:
• **Quotient (Q)** = 000101_2 = **5_{10}**
• **Remainder (R)** = 000000_2 = **0_{10}**
Verification: $50 = 5 \times 10 + 0$, exact match.

(b) (3 pts) Signed Division Sign Rules: Write the fundamental arithmetic identity connecting Dividend (D), Divisor (d), Quotient (q), and Remainder (r). Compute the quotient and remainder for:
(i) $(+23) \div (-5)$ (ii) $(-23) \div (+5)$ (iii) $(-23) \div (-5)$
Explain why computer hardware strictly enforces the rule that "the sign of the remainder must agree with the sign of the dividend."

[Official Solution & Rubric] Rubric: Arithmetic identity 1 pt, Three signed cases 1 pt, Remainder sign rationale 1 pt

1. **Fundamental Arithmetic Identity**:

$$\text{Dividend} = \text{Quotient} \times \text{Divisor} + \text{Remainder} \quad (\text{where } |\text{Remainder}| < |\text{Divisor}|)$$

2. **Three Evaluation Cases**:
• (i) $(+23) \div (-5) \implies \mathbf{q = -4,\ r = +3}$ (Check: $23 = (-4) \times (-5) + 3$)
• (ii) $(-23) \div (+5) \implies \mathbf{q = -4,\ r = -3}$ (Check: $-23 = (-4) \times 5 + (-3)$)
• (iii) $(-23) \div (-5) \implies \mathbf{q = +4,\ r = -3}$ (Check: $-23 = 4 \times (-5) + (-3)$)
3. **Hardware Architectural Rationale**:
Standard integer arithmetic specifies truncation toward zero for integer division. The remainder is defined by residue $r = D - q \cdot d$. To ensure that the magnitude of the product $q \cdot d$ is strictly smaller than or equal to $|D|$ and approaches D from the same direction, **the sign of the remainder must strictly and uniquely match the sign of the dividend (D)**.

[15 Points] Question 7: 16-Bit Half-Precision Range, Precision, and GRS Rounded Floating-Point Addition [Aligned with Exercises 3.27, 3.29]

Consider the IEEE 754 16-bit Half-Precision floating-point standard:

- **Sign bit (S)**: 1 bit
- **Biased Exponent (E)**: 5 bits, with bias value $\text{Bias} = 15$ (Excess-15 system, unbiased exponent $e = E - 15$)
- **Fraction (F)**: 10 bits, normalized significand has an implicit leading 1 ($1.F$)

(a) (5 pts) Dynamic Range and Precision Boundaries (Exercise 3.27 Variant):
(i) (2 pts) Provide the 16-bit binary pattern and decimal value for both the smallest positive normalized number and the largest positive normalized number.
(ii) (1 pt) Determine the machine epsilon (relative precision unit, ϵ_{mach}) of this 16-bit format.
(iii) (2 pts) If the system designer alters the exponent bias to $\text{Bias} = 16$ (Excess-16), analyze the specific changes to the dynamic range and the underflow threshold relative to the standard Excess-15 system.

[Official Solution & Rubric] Rubric: Min/Max normalized 2 pts, Machine epsilon 1 pt, Excess-16 analysis 2 pts

1. **Smallest and Largest Positive Normalized Numbers**:
• Smallest positive normalized: $E = 1$ ($e = 1 - 15 = -14$), $F = 0000000000$.
Binary: `0 00001 0000000000` $\Rightarrow 1.0 \times 2^{-14} = \mathbf{6.1035 \times 10^{-5}}$ (or $1/16384$).
• Largest positive normalized: $E = 30$ ($e = 30 - 15 = 15$), $F = 1111111111$.
Binary: `0 11110 1111111111` $\Rightarrow (2 - 2^{-10}) \times 2^{15} = \mathbf{65504}$.
2. **Machine Epsilon (ϵ_{mach})**:

$$\epsilon_{\text{mach}} = 2^{-10} = \frac{1}{1024} \approx \mathbf{9.7656 \times 10^{-4}}$$

3. **Comparison with Bias = 16 (Excess-16 System)**:
The unbiased exponent range shifts from $[-14, 15]$ to $[1 - 16, 30 - 16] = [-15, 14]$.
• The underflow threshold drops from 2^{-14} to 2^{-15}, allowing smaller positive numbers to remain normalized.
• The maximum representable number decreases from $65504 (\approx 2^{16})$ to approximately $32752 (\approx 2^{15})$, halving the upper dynamic range.

(b) (10 pts) Floating-Point Addition Four Steps and GRS Rounding (Exercise 3.29 Variant):
Perform manual floating-point addition in this 16-bit half-precision format for:
• $A = 26.125_{10}$
• $B = 0.4150390625_{10}$
Demonstrate the full four-step arithmetic derivation:
(i) (2 pts) Convert values A and B into binary scientific notation and specify their 16-bit representations (S, E, F).
(ii) (2 pts) Align binary points: shift the significand of the number with the smaller exponent to the right. State the shift distance and write the aligned significand.
(iii) (2 pts) Extract the exact values (0 or 1) of the **Guard bit (G)**, **Round bit (R)**, and **Sticky bit (S)** generated by the alignment shift.
(iv) (2 pts) Perform significand addition and determine whether post-normalization is required.
(v) (2 pts) Apply the IEEE 754 **Round to Nearest Even** rule to determine whether rounding up occurs. State the final 16-bit binary pattern and its 4-digit hexadecimal machine code representation.

[Official Solution & Rubric] Rubric: Binary conversions 2 pts, Alignment 2 pts, GRS extraction 2 pts, Addition 2 pts, Rounding & Hex 2 pts

1. **Binary Scientific Notation Conversions**:
• $A = 26.125_{10} = 11010.001_2 = +\mathbf{1.1010001000_2} \times \mathbf{2^4}$
$S = 0$; Biased exponent $E = 4 + 15 = 19 = 10011_2$; Fraction $F = 1010001000_2$.
• $B = 0.4150390625_{10} = 0.0110101001_2 = +\mathbf{1.1010100100_2} \times \mathbf{2^{-2}}$
$S = 0$; Biased exponent $E = -2 + 15 = 13 = 01101_2$; Fraction $F = 1010100100_2$.
2. **Binary Point Alignment**:
Exponent difference $\Delta E = 4 - (-2) = \mathbf{6}$ bits. Shift B's significand right by 6 bit positions:
Original significand: `1 . 1 0 1 0 1 0 0 1 0 0`
Shifted right by 6 bits: `0 . 0 0 0 0 0 1 1 0 1 0 [1 0 0 1 0 0]`
3. **GRS Bit Extraction**:
The preserved 10-bit fraction window is `0000011010`.
• Bit 11 is the **Guard bit (G) = 1**
• Bit 12 is the **Round bit (R) = 0**
• Logical OR of all remaining shifted-out bits (bits 13 to 16): $S = 0 \vee 1 \vee 0 \vee 0 = 1 \Rightarrow$ **Sticky bit (S) = 1**
Thus, $GRS = 101_2$.
4. **Significand Addition**:
$$\begin{array}{r@{}l} &1.1010001000\\ + &0.0000011010\text{ (with } GRS=101)\\ \hline &1.1010100010\text{ (with } GRS=101) \end{array}$$ The sum falls in the range $[1, 2)$, so no post-normalization shift is required. Exponent remains $E = 19$.
5. **Round to Nearest Even Evaluation & Final Code**:
Because $G = 1$ and $(R \vee S) = 1$ (i.e., $GRS = 101_2 > 100_2$, strictly greater than half an LSB), **Round Up is mandatory**.
Increment significand LSB by 1: `1010100010 + 1 = 1010100011`.
Assemble 16-bit binary pattern: `0 10011 1010100011`
Convert to 4-digit hexadecimal: `0100 1110 1010 0011` ⇒ **0x4EA3**.

[10 Points] Question 8: IEEE 754 Gradual Underflow, Precision Limits on Integers, and Breakdown of the Associative Law [Aligned with Exercises 3.35, 3.36, 3.37]

Investigate representation limits and computer algebraic properties of the IEEE 754 32-bit Single-Precision format (1 sign bit, 8 exponent bits, 23 fraction bits).

(a) (3 pts) Subnormal Numbers and Underflow Mechanism (Exercise 3.35 Variant):
(i) State the decimal value of the smallest positive normalized number in single precision.
(ii) Explain the architectural rationale for introducing subnormal numbers ($0.F \times 2^{-126}$ with an exponent field of all zeros) to achieve gradual underflow.
(iii) Compute the absolute smallest positive non-zero subnormal number representable in this format.

[Official Solution & Rubric] Rubric: Smallest normalized 1 pt, Gradual underflow rationale 1 pt, Smallest subnormal 1 pt

1. **Smallest Positive Normalized Number**:
$E = 1$ ($e = 1 - 127 = -126$), $F = 0 \Rightarrow 1.0 \times 2^{-126} \approx \mathbf{1.17549 \times 10^{-38}}$.

2. **Architectural Rationale for Gradual Underflow**:
Without subnormal numbers, any result smaller than 2^{-126} abruptly flushes to zero (abrupt underflow), creating an immense representation gap between zero and 2^{-126}. Subnormal numbers replace the implicit leading 1 with 0 ($0.F \times 2^{-126}$), allowing tiny numbers to gracefully approach zero with progressive loss of precision. This guarantees the critical mathematical property that $x - y = 0 \iff x = y$, eliminating unexpected divide-by-zero traps.

3. **Absolute Smallest Positive Non-Zero Subnormal Number**:
$F = 000...001_2 = 2^{-23} \Rightarrow 2^{-23} \times 2^{-126} = \mathbf{2^{-149} \approx 1.4013 \times 10^{-45}}$.

(b) (3 pts) Upper Limit of Consecutive Exact Integers (Exercise 3.36 Variant):
(i) Identify the smallest positive integer N that cannot be represented exactly in IEEE 754 single precision.
(ii) Provide a rigorous mathematical proof based on the 24-bit significand precision (23 fraction bits + 1 implicit leading 1).

[Official Solution & Rubric] Rubric: Exact integer value 1 pt, Precision boundary proof 2 pts

1. **Integer Value**:
The smallest positive integer that cannot be represented exactly is $\mathbf{N = 2^{24} + 1 = 16,777,217}$.

2. **Mathematical Proof**:
• IEEE 754 single precision provides 23 fraction bits plus 1 implicit leading bit, giving a total of **24 bits of significand precision**.
• Any integer whose binary expansion requires ≤ 24 bits can be represented with zero quantization error. Hence, all integers in the closed range $[1, 2^{24}] = [1, 16,777,216]$ are exactly representable without any gap.
• When values enter the interval $[2^{24}, 2^{25}]$, the exponent is 24, making the weight of the LSB equal to $2^{24} \times 2^{-23} = 2^1 = 2$.
• Consequently, in this interval, floating-point numbers can only represent even integers ($16777216, 16777218, \ldots$). The odd integer $2^{24} + 1 = 16,777,217$ falls directly into the gap between representable numbers and rounds to $16,777,216$, making it the first unrepresentable integer.

(c) (4 pts) Breakdown of the Associative Law (Exercise 3.37 Variant):
Given three single-precision floating-point variables: $A = 1.0 \times 10^{30}$, $B = -1.0 \times 10^{30}$, and $C = 1.0$.
(i) Compute the result of $(A + B) + C$ in floating-point hardware.
(ii) Compute the result of $A + (B + C)$ in floating-point hardware.
(iii) Explain why the two evaluations yield completely different results. Why do modern optimizing compilers by default prohibit reordering floating-point operations according to algebraic associativity?

[Official Solution & Rubric] Rubric: Result (i) 1 pt, Result (ii) 1 pt, Explanation & compiler policy 2 pts

1. **Hardware Computation Results**:
• $(A + B) + C = (1.0 \times 10^{30} - 1.0 \times 10^{30}) + 1.0 = 0.0 + 1.0 = \mathbf{1.0}$.
• $A + (B + C) = 1.0 \times 10^{30} + (-1.0 \times 10^{30} + 1.0)$. Because $10^{30} \approx 2^{100}$, aligning $C = 1.0$ with $B = -1.0 \times 10^{30}$ requires shifting C's significand right by approximately 100 bit positions. Since single precision has only 24 bits of significand precision, 1.0 is completely shifted out and lost due to rounding (swallowing/absorption). Therefore, $B + C = -1.0 \times 10^{30}$. Subsequently, $A + B = \mathbf{0.0}$.
2. **Conclusion and Compiler Optimization Policy**:
• Due to finite significand precision and operand scale disparities, **floating-point addition does NOT satisfy algebraic associativity:** $(A + B) + C \neq A + (B + C)$.
• In scientific computing and financial simulations, altering operation grouping can cause catastrophic cancellation or complete loss of significance. Therefore, optimizing compilers adhere strictly to the programmer's explicit expression evaluation tree by default, and never reorder associative operations unless unsafe flags (such as `-ffast-math`) are explicitly supplied by the developer.